

Practical 9

* Aim:- Create a binary search tree (BST) in C++ and perform insert, delete and search operation.

* Theory:- A Binary Search Tree (BST) is a special type of binary tree used for efficient searching, insertion and deletion of elements. In the BST, each node contains a data value and two child pointers: left and right. The BST follows an important property:-

- (i) All values in the left subtree are smaller than the root node.
- (ii) All values in the right subtree are greater than the root node.

Because of this, BST operations can be performed efficiently.

(i) Insertion operation:- Insertion in a BST is done by comparing the value to be inserted with the current node:-

- (a) If the value is smaller, it is inserted into the left subtree.
- (b) If the value is larger, it is inserted into the right subtree. This process continues recursively until the correct position is found.

(ii) Search Operation:- Searching in a BST is efficient because the tree is already ordered:-

- (a) If the required value matches the root, the search is successful.

(b) If the value is smaller, the search continues in the left subtree.

(c) If the value is larger, the search continues in the right subtree.

(iii) Deletion operation :- Deletion in BST is more complex and has three cases :-

(a) Nodes with no child (leaf node) - The node is simply removed.

(b) Node with one child - The node is replaced by its child.

(c) Node with two children - The node is replaced with its inorder successor, and then that successor is deleted.

(iv) Traversal :- Inorder traversal is used to display the BST. It visits the left subtree, root, and right subtree. This traversal prints the elements of the BST in sorted order, which proves that the BST property is maintained.

* Conclusion :- In this practical, a BST was successfully created using C++. The insert, search and delete operations were implemented and tested. The inorder traversal confirmed that the tree maintains elements in sorted order.

BST provides an efficient way to organize and manage data, especially when frequent searching and insertion operations are required.

Practical 10

* Aim :- Implement Depth First Search (DFS) algorithm for a graph using adjacency-matrix representation in C++.

* Theory :- A graph is a non-linear data structure consisting of vertices (nodes) and edges connecting them. Graph traversal is the process of visiting all the vertices of a graph in a systematic way. One of the most important graph traversal techniques is Depth First Search (DFS).

DFS is a traversal algorithm that explores a graph by going as deep as possible along each branch before backtracking. DFS starts from a given vertex and visits one of its adjacent vertices, then continues visiting adjacent vertices of that vertex until no unvisited vertex is left. After reaching a dead end, the algorithm backtracks and explores other paths.

In this practical, the graph is represented using an adjacency matrix, where:-

- (i) $adj[i][j] = 1$ indicates an edge between vertex i and vertex j .
- (ii) $adj[i][j] = 0$ indicates no edge.

A visited array is used to keep track of vertices that have already been visited. This helps in avoiding repeated visits and infinite loop.

The DFS algorithm is implemented using recursion, which internally ~~uses~~ uses a stack to remember the previously visited vertices. The algorithm works as follows:-

- (i) Start from the given starting vertex.
- (ii) Mark the current vertex as visited.
- (iii) Visit all adjacent unvisited vertices recursively.
- (iv) Continue the process until all ~~re~~ reachable vertices are visited.

DFS is commonly used in applications such as path finding, cycle detection, connected components, and topological sorting.

* Conclusion:- In this practical, the depth First Algorithm (DFS) algorithm ~~was~~ was successfully implemented using an adjacency matrix in C++. The program efficiently traverses all connected vertices starting from a given vertex. DFS is a useful technique for exploring graph structures and solving connectivity-related problems.